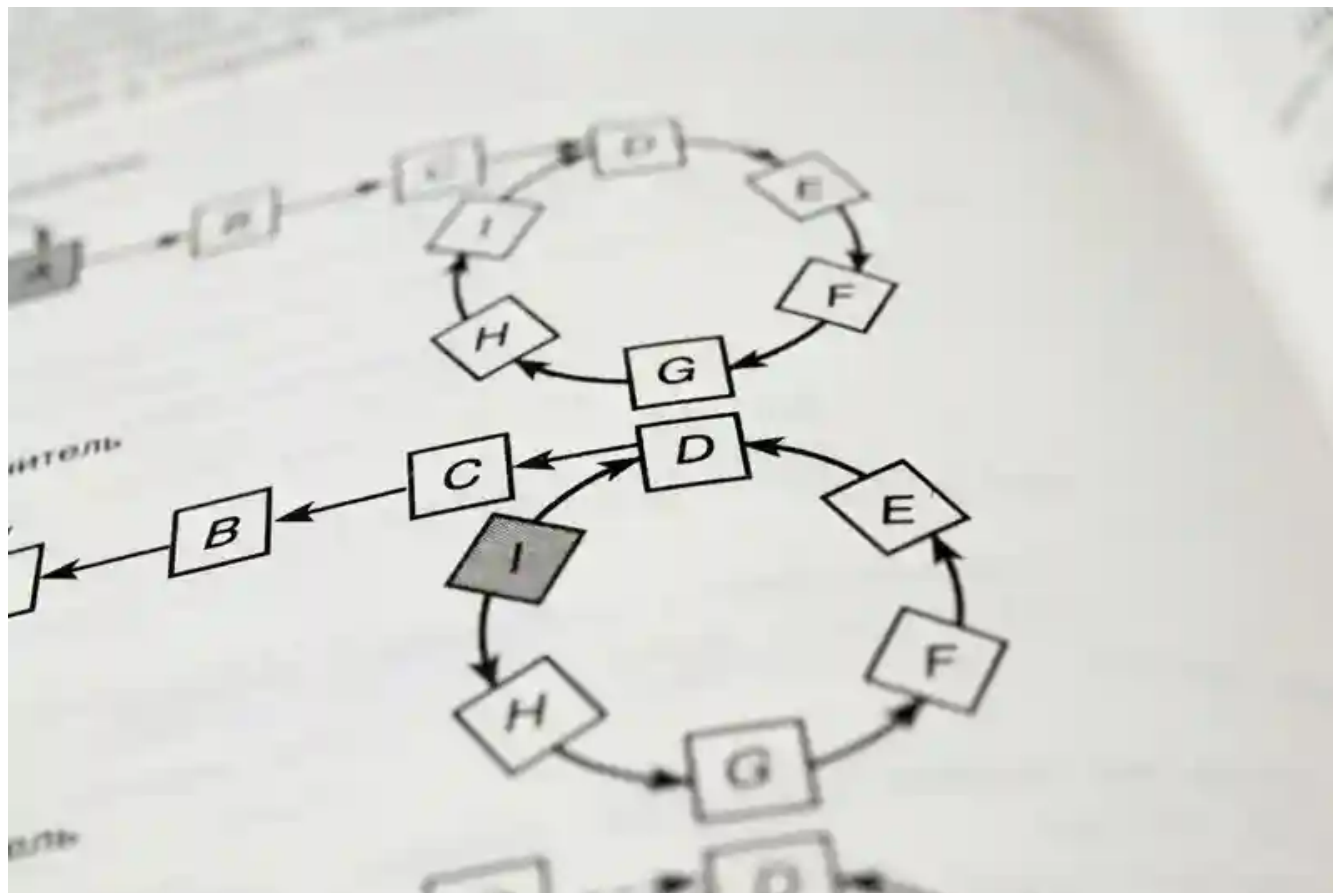


# Rsync 算法简介

victorchutian 收录于 类别 [Algorithm](#)

📅 2026-05-20 🔗 2026-05-20



## 目录

- | 传统方案的困境
- | Rsync 算法核心流程
  - | 第一步：分块
  - | 第二步：计算校验和
  - | 第三步：发送校验和列表
  - | 第四步：搜索匹配块
  - | 第五步：构造指令流
- | 滚动校验和：算法的灵魂
  - | 数学定义
  - | 具体数值示例
  - | 滚动更新公式
- | 三层校验和搜索
  - | 第一层：哈希表查表
  - | 第二层：校验和比对
  - | 第三层：强校验和确认
- | 简化流程图

| 匹配后如何处理

| 流水线处理

| 实验数据

| 总结

| 关键符号表

你有没有遇到过这种情况：服务器上有两份代码，版本 A 和版本 B，你只想把 A 的改动同步到 B，但网络带宽只有几 Mbps，传输一个 24MB 的文件要等上好几分钟。

也许你会想到先在本机 diff，然后把 patch 发过去。没错，这是个思路。但你有没有想过：如果 A 和 B 都不在同一台机器上，diff 根本跑不起来怎么办？

这就是 rsync 要解决的问题。

## | 传统方案的困境

让我们先理清场景：机器  $\alpha$  有文件 A，机器  $\beta$  有文件 B，两者通过网络相连。我们想把 B 更新成 A 的副本。

最直接的办法是直接把 A 发过去，但如果 A 很大呢？压缩一下？普通压缩顶多省 2-4 倍的传输量，依然很慢。

你可能会想到：如果 A 和 B 很相似（比如同一份源码的不同版本），我能不能只发“差异”过去？这就是经典的 diff/patch 思路。GNU diff 生成的 patch 文件通常比原文件小得多，发送效率更高。

但这里有个致命问题：diff 需要同时看到两个文件才能工作。如果 A 和 B 在不同的机器上，你就得先把其中一个传过去——传完之后还需要 diff 吗？

rsync 算法正是在这个背景下诞生的：在不同时拥有两个文件的情况下，高效计算它们之间的差异。

## | Rsync 算法核心流程

rsync 的核心思想很巧妙：发送方告诉接收方“你已经有了哪些块”，而不是“你需要什么”。

假设机器  $\beta$ （接收方）已有文件 B，想同步成 A。整个过程分五步走：

### | 第一步：分块

$\beta$  将文件 B 切分成固定大小的非重叠块，块大小记为 S 字节。最后一块可能不足 S 字节。

### | 第二步：计算校验和

对 B 的每个块， $\beta$  计算两个校验和：

- 弱校验和：32 位“滚动校验和”（rolling checksum），计算成本极低
- 强校验和：128 位 MD4 校验和（rsync 后续版本也支持 MD5、SHA-1 等更安全的选项），碰撞概率极低

⚠️ **安全提示：** MD4 本身已不推荐用于安全场景（如数字签名、密码存储），但在 rsync 中仅用于数据完整性校验，碰撞攻击并非威胁模型的一部分。

## 第三步：发送校验和列表

$\beta$  将所有块的校验和（弱 + 强）发送给  $\alpha$ 。这一步传输量很小——每个块只需要 20 字节（4 字节弱校验和 + 16 字节 MD4）。

## 第四步：搜索匹配块

$\alpha$  拿到校验和列表后，在文件 A 的所有偏移位置（而不只是块边界）滑动窗口，检查每个长度为 S 的子串是否与 B 中某个块的校验和匹配。

这是算法的关键——我们后面会详解如何高效实现这个“全量搜索”。

## 第五步：构造指令流

$\alpha$  向  $\beta$  发送一系列指令，构造出 A 的副本：

- **引用指令：** 指向 B 中某个匹配块的索引
- **字面数据：** A 中无法匹配的字面字节流

$\beta$  收到指令后，本地组装出 A——匹配的块直接复用 B 中的数据，不匹配的则写入  $\alpha$  发来的字面数据。

**最终效果：** 只有 A 中真正新增或修改的部分才需要传输，网络传输量大幅减少。而且整个过程只需要一次往返，最小化了网络延迟的影响。

## 滚动校验和：算法的灵魂

整个 rsync 算法最精妙的设计，是那个“计算成本极低”的弱校验和——滚动校验和（Rolling Checksum）。

为什么叫“滚动”？因为它支持  $O(1)$  增量更新：已知位置  $k$  到  $l$  的校验和，可以直接算出位置  $k+1$  到  $l+1$  的校验和，无需重新扫描。

这个特性是 rsync 高效搜索的基础。

## 数学定义

rsync 的滚动校验和由两部分组成：

$$a(k, l) = \left( \sum_{i=k}^l X_i \right) \bmod 2^{16}$$
$$b(k, l) = \left( \sum_{i=k}^l (l - i + 1) X_i \right) \bmod 2^{16}$$

$$s(k, l) = a(k, l) + 2^{16} \cdot b(k, l)$$

其中  $X_k \dots X_l$  是字节序列，最终的校验和  $s(k, l)$  是一个 32 位整数。

## | 具体数值示例

为了帮助理解，我们手算字节序列 `['h','e','l','l','o']`（ASCII 码：104, 101, 108, 108, 111）的校验和，窗口大小  $S=4$ 。

窗口 `[0,3] = "hell"`：

- $a = (104 + 101 + 108 + 108) \bmod 65536 = 421 \bmod 65536 = 421$
- $b = (4 \times 104 + 3 \times 101 + 2 \times 108 + 1 \times 108) \bmod 65536 = 1085 \bmod 65536 = 1085$
- $s = 421 + 65536 \times 1085 = 71,094,181$

滚动到窗口 `[1,4] = "ello"`（移出 'h'，加入 'o'）：

- $a' = (421 - 104 + 111) \bmod 65536 = 428$
- $b' = (1085 - 4 \times 104 + 428) \bmod 65536 = 817$
- $s' = 428 + 65536 \times 817 = 53,541,180$

可以看到，只需要  $O(1)$  操作就完成了校验和更新，而无需重新扫描 4 个字节。

初始窗口 "hell" 的校验和计算：

```
text
a = X0 + X1 + X2 + X3 = 104 + 101 + 108 + 108 = 421
b = 4×X0 + 3×X1 + 2×X2 + 1×X3 = 416 + 303 + 216 + 108 = 1085
s = 421 + 65536 × 1085 = 421 + 71,056,760 = 71,094,181
```

滚动到 "ello" 时的增量计算：

```
text
a' = (a - X0 + X4) mod M = (421 - 104 + 111) mod 65536 = 428
b' = (b - 4×X0 + a') mod M = (1085 - 416 + 428) mod 65536 = 817
s' = 428 + 65536 × 817 = 428 + 53,541,632 = 53,541,180
```

验证：直接计算 "ello" 的校验和 =  $428 + 65536 \times 817 = \mathbf{53,541,180}$  ✓

```
text
a = 101 + 108 + 108 + 111 = 428
b = 4×101 + 3×108 + 2×108 + 1×111 = 404 + 324 + 216 + 111 = 817
```

## | 滚动更新公式

有了这个定义，滑动窗口时可以用以下递推公式  $O(1)$  更新校验和：

$$a(k+1, l+1) = (a(k, l) - X_k + X_{l+1}) \bmod 2^{16}$$

$$b(k+1, l+1) = (b(k, l) - (l - k + 1)X_k + a(k+1, l+1)) \bmod 2^{16}$$

简单来说：**移出一个字节，加入一个新字节，就能直接算出新窗口的校验和**。这意味着扫描整个文件 A 的所有位置，校验和计算的总复杂度是  $O(N)$ ，几乎不需要额外开销。

rsync 的弱校验和设计灵感来自 Mark Adler 的 adler-32，但做了一些调整。实际使用中，它的**误报率（false alarm rate）极低**：实验数据表明，真实匹配数超过误报数 1000 倍以上。

## | 三层校验和搜索

拿到滚动校验和后，a 怎么快速找到 A 中与 B 匹配的块？

### | 第一层：哈希表查表

rsync 维护一个  $2^{16}$  条目的哈希表，按校验和的 16 位哈希值组织。B 的所有块校验和按哈希值排序后存入。

对 A 的每个位置，先算出 32 位滚动校验和及其 16 位哈希，查哈希表——如果对应位置为空，说明这块肯定不匹配，跳过。

### | 第二层：校验和比对

如果哈希命中，则遍历该哈希桶内的所有条目，逐个比对 32 位滚动校验和。

### | 第三层：强校验和确认

滚动校验和匹配后，还需要计算 A 对应位置的 MD4 强校验和（rsync 后续版本也支持 MD5、SHA-1 等），与桶内条目的强校验和比对。强校验和匹配才视为真正匹配——这一步排除了几乎所有误报。

## | 简化流程图

text

在文件 A 的每个偏移位置 i：

```
|
| 计算滚动校验和 r(i)
|   └ 取 r(i) 的低 16 位查哈希表
|       └ 哈希表为空 → 跳过（不可能匹配）
|       └ 哈希命中 → 进入下一层
|           |
|           └ 逐个比对 32 位滚动校验和
|               └ 不匹配 → 跳过
|               |
|               └ 滚动校验和匹配 → 计算强校验和
|                   └ 强校验和不匹配 → 误报，跳过
|                   └ 强校验和匹配 → ✅ 找到匹配块！
|                           发送 "引用指令 + 字面数据"
```

每层都在前一层的输出基础上进一步过滤，直到确认真正匹配。

三层策略的核心逻辑：用最小的计算成本过滤掉不可能匹配的位置，把昂贵的 MD4 计算压缩到最少。

## | 匹配后如何处理

找到一个匹配后， $\alpha$  立即将以下内容发给  $\beta$ ：

- 1. 当前偏移到上一次匹配之间的字面数据
- 2. 匹配的 B 块索引

发送完匹配信息后，搜索从匹配块末尾重新开始。这个策略对“两文件几乎相同”的常见场景极为高效——省去了大量重复的无效搜索。

## | 流水线处理

如果要同步多个文件，rsync 还有一招——**流水线 (pipelining)**。

具体做法： $\beta$  启动两个独立进程：

- 一个负责生成并发送校验和列表给  $\alpha$
- 另一个负责接收  $\alpha$  返回的差异信息并重建文件

只要网络链路是全双工的，这两个进程可以并行运行，网络链路在双向都能保持高利用率。对于高延迟链路（如卫星链路或跨国网络），流水线化带来的延迟隐藏效果非常显著。

## | 实验数据

1998 年 Tridgell 和 Mackerras 在论文中给出了真实测试结果。

**测试场景：**Linux 内核源码 tar 包，1.99.10  $\rightarrow$  2.0.0 版本。原始文件约 24MB，版本间有 291 个文件变化、19 个文件删除、25 个文件新增。GNU diff 输出 2.1MB。

不同块大小下的 rsync 传输量：

| 块大小（字节） | 匹配次数   | 传输数据量（字节） | 写入量       | 读取量（校验和）  |
|---------|--------|-----------|-----------|-----------|
| 300     | 64,247 | 5,312,200 | 5,629,158 | 1,632,284 |
| 500     | 46,989 | 1,091,900 | 1,283,906 | 979,384   |
| 700     | 33,255 | 1,307,800 | 1,444,346 | 699,564   |
| 900     | 25,686 | 1,469,500 | 1,575,438 | 544,124   |
| 1100    | 20,848 | 1,654,500 | 1,740,838 | 445,204   |

可以看到，块大小 500 字节时，只传输了约 1MB 数据，不到原文件大小的 5%，远优于 2.1MB 的 diff 文件。

另一个测试来自 Samba 源码（1.7MB），块大小 700 字节时传输了 195KB，而 GNU diff 是 120KB——对这种小文件来说 rsync 略有额外开销，但换来了无需预先同步文件的巨大灵活性。

误报率方面，块大小  $\geq 500$  字节时，false alarm 数量降至 100 以下，相比数万次真实匹配几乎可以忽略不计。

## | 总结

rsync 算法的核心贡献在于：在不同时拥有两个文件的前提下，高效计算同步所需的最小差异集。这解决了远程文件同步的核心痛点。

它的关键设计点：

- **滚动校验和**：O(1) 滑动窗口计算，使得全量块匹配搜索成为可能
- **双层校验和**：弱校验和快速过滤，强校验和（MD4）精确确认
- **哈希分层查找**：将搜索范围逐层压缩，避免无谓的 MD4 计算
- **一次往返**：最小化高延迟链路的影响
- **流水线处理**：多文件同步时可充分隐藏网络延迟

rsync 的适用范围很广：代码同步、备份系统、镜像分发、增量传输等。唯一需要注意的是，块大小的选择会影响传输效率——太大则匹配精度下降，太小则校验和列表膨胀。一般 500-1000 字节是常见的经验值。

如果你想深入研究原始论文，可以查看 [rsync 官方技术报告](#)，作者是 Andrew Tridgell（rsync 的发明人）和 Paul Mackerras。

## | 关键符号表

| 符号        | 含义                      |
|-----------|-------------------------|
| $X_i$     | 文件的第 $i$ 个字节            |
| $k, l$    | 滑动窗口的起始/结束索引            |
| $a(k, l)$ | 弱校验和的第一部分（16 位）         |
| $b(k, l)$ | 弱校验和的第二部分（16 位）         |
| $s(k, l)$ | 最终的 32 位滚动校验和           |
| $S$       | 块大小（字节）                 |
| $M$       | $2^{16}$ （65536），模运算的基数 |
| $\alpha$  | 发送方机器（持有文件 A）           |
| $\beta$   | 接收方机器（持有文件 B）           |

更新于 2026-05-20

---

 Sync